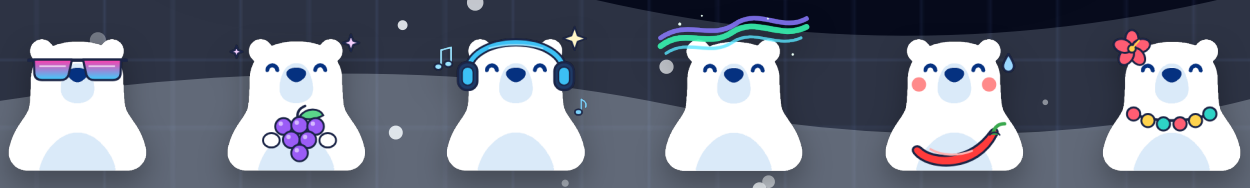
nanuq

TECHNICAL MANUAL

How Nanuq is built

Architecture with diagrams, each platform inside out and, at the end, every function in the code.





Version 1.47.3 · November 12, 2026 · English · Free software (GPL-3.0-or-later)



Contents

Part 1 explains the design; part 2 is the reference for every function.

ARCHITECTURE

1 **Overview** 6
One interface, three wrappers.

2 **The repository** 8
What lives in each folder.

3 **The page** 9
How app.js starts, navigates and reacts.

4 **Data and storage** 11
Anything about you is encrypted.

5 **Playback** 13
Where each song comes from and what happens when something fails.

6 **Library and Jellyfin** 15
The API Nanuq uses and what it builds on its own.

7 **DJ Nanuq** 16
Rule-based rounds, no AI.

8 **My devices** 17
One Nanuq plays and the others control it, through Jellyfin.

9 **The party** 18
Phones sending songs to a computer over the local network.

10 **The Android app** 20
WebView + native Java.

11 **The desktop app** 21
Electron for Linux and Windows.

12 **Security** 22
Nothing from outside is trusted.

13 **Building and releasing** 23
One command for every platform.

14 **Conventions and pitfalls** 24
What to know before changing anything.

15 **License** 25
GPLv3 or later, file by file.

DEEP LEVEL: EVERY FUNCTION

16 **web/app.js** 28
603 functions, in file order.

Base: helpers and storage 28
Server addresses 29
Jellyfin API 29
Audio: waveforms and synthesizer 31
Demo library 31
Settings and themes 33
Downloads (Android) 33
State and routing 34
Albums from tags 34
Lyrics search 35
Rendering helpers 35
Views 35
DJ Nanuq 36
Your year and Your week (video) 38
Song requests (SomeDL) 39
Player 40
Lyrics 43
Queue 44
Toasts, menus and dialogs 44
My devices 44
Party (watch party) 46
Global actions 48
Login 48

17 **Rest of the page** 49
Languages and service worker.

web/i18n.js 49

web/sw.js 49

18 **Android** 50
Native Java and the JavaScript bridge.

android/app/src/main/java/com/nanuq/app/Discovery.java 50
android/app/src/main/java/com/nanuq/app/Downloads.java 50
android/app/src/main/java/com/nanuq/app/LocalServer.java 51
android/app/src/main/java/com/nanuq/app/Page.java 52
android/app/src/main/java/com/nanuq/app/MainActivity.java 53
android/app/src/main/java/com/nanuq/app/MediaCenter.java 53
android/app/src/main/java/com/nanuq/app/MediaButtonReceiver.java 54
android/app/src/main/java/com/nanuq/app/Resume.java 55
android/app/src/main/java/com/nanuq/app/NanuqBrowserService.java 55
android/app/src/main/java/com/nanuq/app/NativeHttp.java 55
android/app/src/main/java/com/nanuq/app/PlaybackService.java 55
android/app/src/main/java/com/nanuq/app/Prefetch.java 55
android/app/src/main/java/com/nanuq/app/SecureStore.java 56
android/bridge/android-bridge.js 57
android/build-apk.sh 57
android/app/src/main/java/com/nanuq/app/OggCheck.java 57
android/app/src/main/java/com/nanuq/app/NativeAudio.java 57

19 **Desktop** 59
Electron and its packaging.

linux/main.js 59
linux/preload.js 61
linux/prepare-web.js 61
linux/collect.js 61
linux/flatpak/build-flatpak.sh 61

20 **Build** 62
The script that builds everything.

scripts/build.sh 62

PART 1

Architecture

How Nanuq is built: a single web interface, the apps that wrap it and how it talks to Jellyfin, to your other devices and to the party.

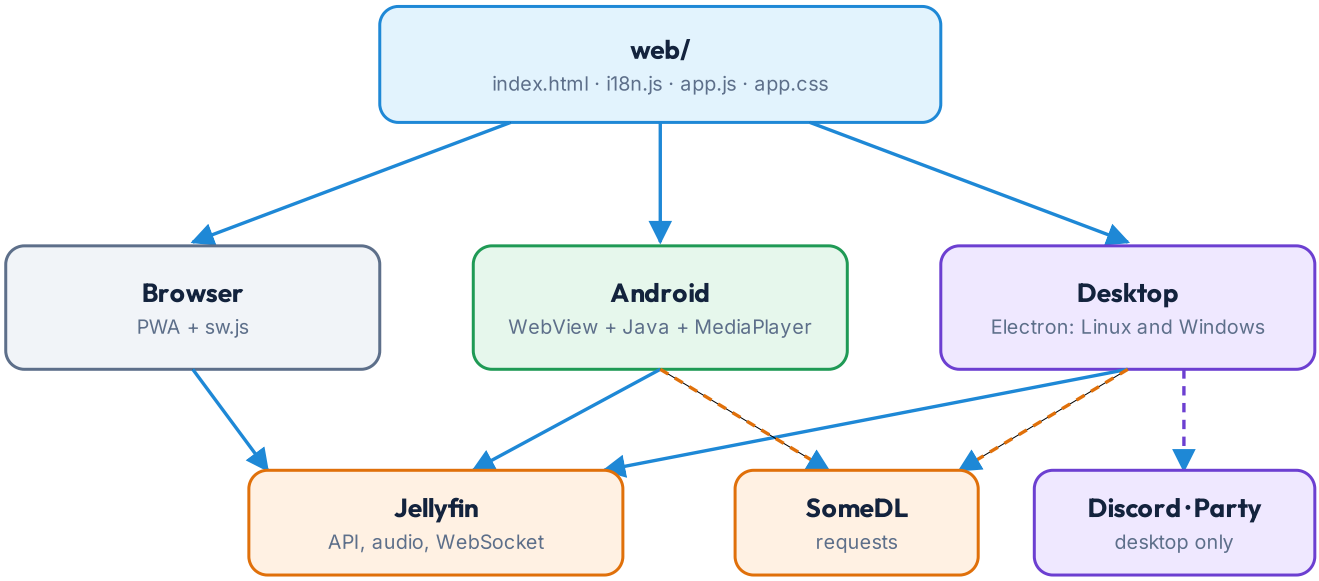


01 Overview

One interface, three wrappers.



All of Nanuq's interface lives in `web/`: four files, with no build step and no dependencies. The Android and desktop apps have no interface of their own: they copy that same folder at build time and add, through a “bridge”, what a web page can't do on its own.



Every platform serves the same page; dashed arrows are what only native code does (SomeDL without CORS, Discord, the party room).

Why this way

- **One change reaches every platform**: the interface is only edited in `web/`; Android and Linux copy it at build time (`syncWeb` in Gradle, `prepare-web.js` in Electron).
- **Secure context without HTTPS**: Android serves the page at `http://localhost` and Electron at `http://127.0.0.1:47863`. That gives `crypto.randomUUID` and storage (the service worker is only used in the browser), while still talking to an `http://` Jellyfin on the local network without mixed-content blocks.
- **No third-party services**: Nanuq only talks to your Jellyfin, your SomeDL and (if you enable it) the Discord app on your computer. The only external requests are the Google Fonts.

Ports

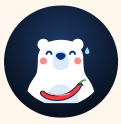
Port	Who	What for
TCP 47863	Desktop (127.0.0.1)	Serves the page and, with the party open, the room's audio (`/fiesta/<token>/…`). Fixed port because storage depends on the origin.
TCP 47864	Desktop (local network)	Party WebSocket (`/fiesta`) and covers for guests. Only with the room open and only private addresses.
UDP 47865	Desktop	Answers “Who is NanuqParty?” so phones find the room (≤ 30 answers/s).
UDP 7359	Jellyfin	Official server discovery (“Who is JellyfinServer?”), from Android and desktop.
TCP random	Android (127.0.0.1)	`LocalServer` : downloads, covers and prefetched songs, with a token in the path.
8096 / 8920	Jellyfin	Ports assumed if you type an IP or short name without a port.
5000	SomeDL	Port assumed for SomeDL without a port.

02 The repository

What lives in each folder.



Path	Contents
`web/index.html`	Structure, strict CSP and SVG `<defs>`: `#i-*` icons and the vector bear (`#djb-*`).
`web/i18n.js`	Languages: `LANGS`, `LANG`, `LOC`, the `EN` dictionary (key = Spanish text) and `tx()`. Loaded before app.js.
`web/app.js`	All the logic (6,108 lines). `const VERSION` is the source of truth for the version.
`web/app.css`	All styling, with per-theme variables (18 themes).
`web/sw.js`	Service worker, browser only: caches the app (network first for code), never music or API. Android and desktop don't register it.
`android/`	WebView + Java (`com.nanuq.app`), the `bridge/android-bridge.js` bridge, `build-apk.sh` and its tests (`test/`: OggCheck and the bridge player).
`linux/`	Electron: `main.js`, `preload.js`, `prepare-web.js`, `collect.js`, `flatpak/` and `test/` (`main.js` tests). It also builds the Windows installer.
`scripts/build.sh`	Bumps the version and builds everything into `dist/<version>/`.
`docs/`	README images and these manuals (`docs/manuals/src`: `node docs/manuals/src/build.js`).



Heads-up

`dist/`, `keystore.properties` and `*.jks` files never go into the repository. The APK signing key lives outside (`~/.android/nanuq-release.jks`): without it, updates can't be published.

03 The page

How app.js starts, navigates and reacts.



Startup

1. On load, `i18n.js` picks the language and `txStatic()` translates the fixed HTML texts.
2. `SEC` detects the native encrypted store and `secMigrate()` moves there whatever older versions left in plain text.
3. The boot block draws the sidebar and tabs, applies the theme and starts downloads (`DL.init`).
4. `Vault.init()` retries for a few seconds if the Keystore or keyring isn't ready yet; then `Vault.load()` reads the session.
5. With a session: `startApp(new Jellyfin(sess))` restores the queue and position (`QPOS`), restores the DJ and opens Home. Without one: `showLogin()`.

Navigation

`S.stack` is the screen stack. `go(name, params, opts)` pushes a screen (the `TOP` ones clear the stack) and calls `render()`, which runs `V[name](box, params)`. Each view is an async function that fills its box; if it leaves a `_cleanup`, it's called on exit. `back()` first closes whatever is on top (menu, lyrics editor, player) and then goes back. With the party open (`party-host` / `in-party`) only the party screen exists.

Events

The CSP forbids inline scripts, so there's no `onclick=`: every button has `data-act="…"` and a single global click handler decides what to do (“Global actions” section). Images that must disappear when they fail carry `data-rm`.

Languages

Every visible text is written in Spanish inside `tx('…')`; `i18n.js` holds the translation keyed by the Spanish text. With values: `tx('… {0} …', [x])`. Switching language saves the position and reloads the page, so texts built at load time are translated too.

Sections of app.js

Section	From line
Base: helpers and storage	1
Server addresses	122

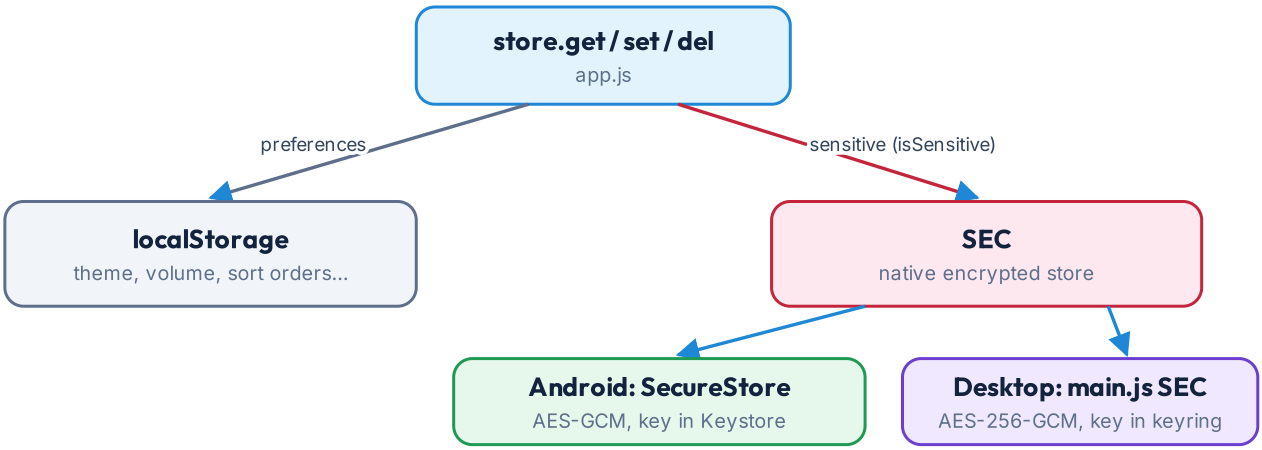
Section	From line
Jellyfin API	177
Audio: waveforms and synthesizer	489
Demo library	550
Settings and themes	748
Downloads (Android)	808
State and routing	967
Albums from tags	1017
Lyrics search	1122
Rendering helpers	1220
Views	1250
DJ Nanuq	1317
Your year and Your week (video)	2277
Song requests (SomeDL)	2988
Player	3279
Lyrics	3863
Queue	4209
Toasts, menus and dialogs	4283
My devices	4294
Party (watch party)	4992
Global actions	5733
Login	5894
Boot	6073

04 Data and storage

Anything about you is encrypted.



`store` is the only way to storage. Interface preferences (theme, volume, sort orders) go to `localStorage` with the `nanuq.` prefix; anything that says something about you goes to the platform's encrypted store. Since preferences aren't encrypted, they're validated when read (right after `Settings`): an altered or outdated value goes back to the default.



In the browser there is no secure store: everything stays in localStorage (Settings says so).

Key	What it holds
session	Jellyfin server, user and token.
queue · queuePos	Your queue (up to 3000) and the second you were at (queuePos isn't sensitive: just a random id and seconds).
libIdx · lyrIdx	Indexes of your library (albums from tags) and of your lyrics. Each in its own encrypted file (`datos-<key>.bin`, same key): if they went with the rest, every song change would re-encrypt and write several MB.
dj · djHist	Current DJ session and history of what it played.
plays · mp3ids	Plays per song on this device and songs that go as MP3.
somedl · somedlReqs · somedlLast	SomeDL server and your requests.
servers · lastServer · device · party	Recent servers, device id and the current party (phone).
lrc.<id>	Lyrics saved only on this device.

Key	What it holds
native.resume	Android (written by Java, not the page): the last song for the “resume” card (`Resume`); its cover goes in private storage. Deleted on logout.

Store states

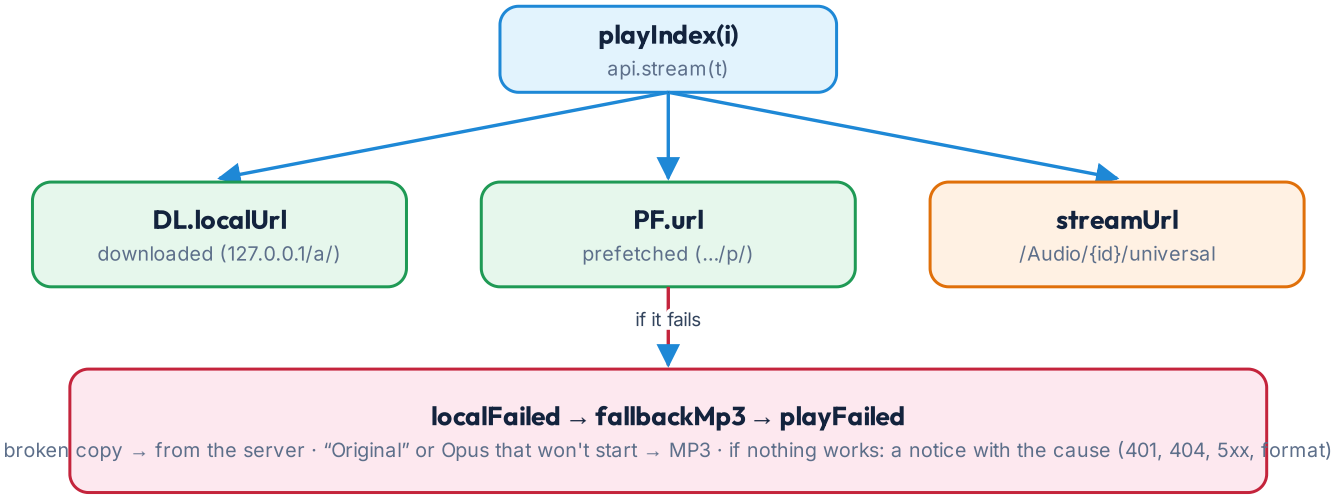
`SEC.kind` can be the store's name (works), `locked` (encrypted data that won't open right now: retried), `lost` (the key is gone: “Start over” is offered, which sets the data aside without deleting it) or `null` (no store). While it isn't open, new data lives only in memory (`SEC.mem`): it's **never** written in plain text.

05 Playback

Where each song comes from and what happens when something fails.



`P` holds the queue, the index and two players (`P.els`, created with `newAudio()`; `P.audio` is always the current song's). In the browser and on desktop they are `<audio>`; on Android, `NanuqNativeAudio`, used the same way but playing through the system player. `playIndex(i)` picks the source with `api.stream()`: first the downloaded copy, then the prefetched one and, if neither exists, Jellyfin streaming.



On Android, the first two sources are served by the phone itself; on Android and desktop the page never loads the whole audio into memory.

Details that matter

- **Seeking in live transcodes**: if Jellyfin transcodes, the file can't be seeked; `seekTo` requests the audio again with `StartTimeTicks` and remembers the offset (`P.off`). While dragging, the request goes out on release.
- **Stalls**: `armStallWatch` warns if it doesn't start within 10 s; `stallCheck` resumes from the same second if the network dropped mid-song; `netBack` does it when the network returns. A network error with the song already playing doesn't switch to the MP3 fallback: it just waits for the network.
- **Quality**: 96 kbps by default; `qualPaint` shows it in the maximized player ("Opus · 96 kbps", "Original", "On the phone").
- **Formats**: `streamUrl` lets FLAC, Opus, AAC, MP3, WAV and OGG play as-is (in "Original" or under the bitrate); if it must convert, it asks for Opus in Ogg (`Settings.codec`, if the player opens it: `OPUS_OK`) or MP3. Anything that failed once goes as MP3 (`MP3.ids`). Downloads use the same format.
- **Fades**: `pauseSoft` / `playSoft` with `ramp()`, linear in decibels (−50 dB = silence), from 0.5 to 5 s (`Settings.fadeMs`). While fading out it already counts as paused (`FADE.out`) and ▶ reverses it. The

restored volume is always yours (`userVol`). Dragging the progress bar uses 30–40 ms ramps only against clicks (`P.scrub`).

- **DJ crossfade**: `xfCheck` on `timeupdate`; 3–5 s before the end (`Settings.xfade`), `xfStart` swaps the two `<audio>` and crosses them with constant-power (sine/cosine) curves. Events from the outgoing one are ignored (`onAudio`); `xfKill` stops it on pause or song change.
- **Foreign pause**: if another app took the audio output (`P.extPause`), resuming reloads the song so it doesn't stay silent (doesn't apply to the Android player).
- **Screen off** (`pageHidden`, browser and Android only): no fades or DJ crossfade, because their timers are throttled; if hidden mid-ramp, `rampsFinish` completes it. The desktop app isn't throttled when minimized.
- **System controls**: `navigator.mediaSession`; on Android the native bridge replaces it. On desktop a silent audio (`rcKeep` / `sessionHold`) keeps the controls while loading or while you control another device.
- **Favorites**: Jellyfin 12 overwrites the favorite of the playing song with every playback report; `FAVHOLD` saves it again after each report.

06

Library and Jellyfin

The API Nanuq uses and what it builds on its own.



Jellyfin route	What for
/System/Info/Public · /Users/AuthenticateByName	Recognize the server and log in.
/Items · /Artists/AlbumArtists · /MusicGenres · /Playlists	Library lists (with several fallback routes depending on the version).
/Audio/{id}/universal	The audio (streaming, downloads, prefetch, party).
/Audio/{id}/Lyrics · .../RemoteSearch/Lyrics	Read, upload and search lyrics (10.9+).
/UserFavoriteItems/{id}	Favorites (and the pre-10.9 route as fallback).
/Items/{id}/InstantMix	Song radio.
/Sessions · /Sessions/{id}/Playing · /socket	My devices: sessions, commands and live events.
/Sessions/Playing(/Progress|/Stopped)	Playback reports (what the server sees).
/DisplayPreferences/nanuq-dj	DJ state shared between your devices.
/user_usage_stats/...	Playback Reporting plugin for Your week / Your year (if present).

Albums from tags (LIB)

Jellyfin builds albums by folder; Nanuq builds them from each song's “Album” + “Album artist” tags (`albumKeyOf`), joins “(Disc 2)” with its album and removes duplicates. The index of all your songs is fetched 1000 at a time (three pages at once), stored encrypted and refreshed every 10 minutes. These album ids start with `m` (`albumGid`).

Lyrics search (LYRX)

Jellyfin doesn't search inside lyrics: Nanuq indexes the songs with `HasLyrics` (3 at a time, up to 8000) plus local lyrics, normalized without accents or punctuation (`lyrNorm`). Searches need 4 characters or more.

07 DJ Nanuq

Rule-based rounds, no AI.



`DJ.next()` shuffles round types by weight, avoids repeating recent ones and tries until it finds one that **explores** (songs that didn't play in recent weeks). Each song carries its round in the queue id: `dj—<session>-<round>-<random>`.

Round	Weight	What it plays
genre	6	A genre from your library (≥ 3 songs in a 400-song sample), not repeating recent ones.
decade	3.5	A real decade from your library; past ones weigh ×1.5.
discover	3	Never played, first in your favorite genres.
forgotten	3	More than 45 days without playing.
mood	1	By time of day (calm morning, energetic afternoon…).
fresh · fav · recent · top	.8 · .6 · .15 · .15	Newly added, favorites, recent days and most played (with cooldown).

`DJ.rank()` decides how “worn out” a song is: anything from the last 3 days or from recent DJ sessions barely gets in; forgotten songs go first. When music moves to another device, `DJSHARE` leaves the state in your Jellyfin and the other one adopts it (`DJ.adopt`); whatever is read is cleaned with `djClean`. While the first round is built, `DJL` shows Nanuq at the turntable with lines (`DJ_LOAD_SAYS`) until it plays.

08 My devices

One Nanuq plays and the others control it, through Jellyfin.





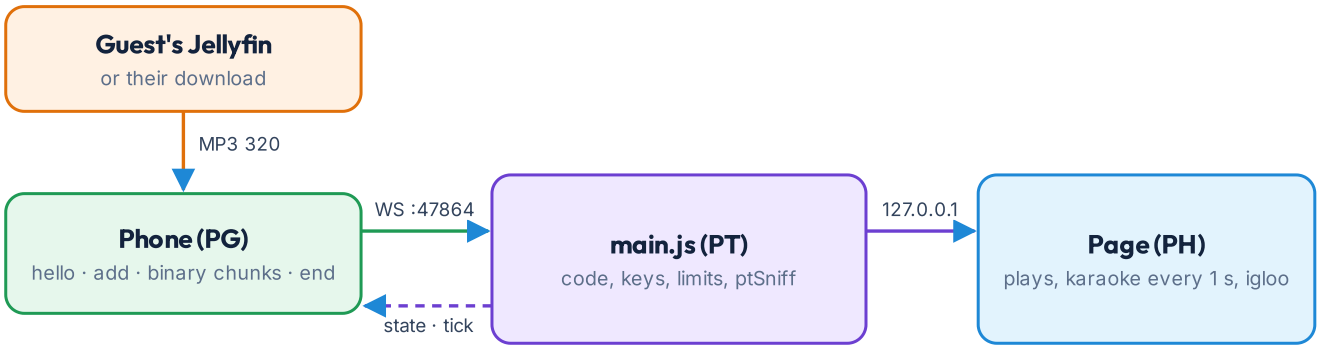
No port is opened on the devices: everything goes through your Jellyfin, with your session.

- **Who shows up**: sessions with client “Nanuq”, of your same user, active within 15 minutes by the *server's* clock (skewed clocks don't matter).
- **Same network**: same /24 in IPv4 or /64 in IPv6 (or both private), using the address the server sees.
- **Accepted commands**: `Play` and `GeneralCommand` only if your user sent them (data Jellyfin fills in); `Playstate` is filtered by Jellyfin permissions.
- **While controlling**, your local queue is set aside (`RC.localQ`) and comes back when you disconnect; the position is estimated from the last report (`RC.anchor`).
- On Android, with the screen locked, the native app calls `__nanuqTick` every 20 s to keep the WebSocket alive.

09 The party

Phones sending songs to a computer over the local network.





UDP :47865 "Who is NanuqParty?" → room name and port

The phone fetches the song from ITS Jellyfin and hands it to the computer in 256 KB chunks, without holding it whole in memory.

Message	Direction	What it is
hello {code|key, name, theme}	phone → PC	Join with the 6-digit code or come back with its key.
welcome {you, key, room, chunk}	PC → phone	Joined: its key to come back without the code (2 h).
add · [id + bytes] · end · remove	phone → PC	Send a song (data, lyrics, cover) and its bytes.
addOk · addErr · ready	PC → phone	Answer to the request and notice that it's ready.
state · tick	PC → phones	List, guests and what's playing; second and karaoke line.
bye {why} · leave · ping	both	Leave, be removed or say it's still there.

Limits and defenses

`PT_MAX` in `main.js`: `{ guests: 12, conns: 24, file: 80e6, disk: 800e6, list: 60, perGuest: 10, frame: 1 << 20, text: 450e3, lyrics: 200e3, cover: 150e3 }`. Private addresses only (`ptLocal`), ≤ 4 not-yet-identified connections per address, 5 wrong codes per minute per address and, after 30 failures overall, a new code and one minute closed. JSON is read with `jparse` and texts with `ptStr`; a message that breaks something only drops that connection.

Songs on disk, encrypted

Songs go to a folder on disk (`ptBase()`: `~/.cache/nanuq/nanuq-fiesta-*` on Linux; in the Flatpak, inside `~/.var/app/com.nanuq.app/cache`), not to the temp folder, which is RAM there. A song is refused if less

than 1 GB is free. Each song is stored **encrypted and authenticated** with AES-256-GCM in 64 KB blocks (`PT_CH`): the key is random and lives only in the room's memory (`PT.fKey`), and the nonce is 8 random bytes per song plus the block number (`ptNonce`). `ptSeal` encrypts on arrival; `ptServe` checks the exact size (`ptEncSize`) and `ptOpenRange` decrypts only the blocks of the requested range. If another app changes a byte, truncates the file or swaps blocks, that block isn't served and the song leaves the list.

10 The Android app

WebView + native Java.

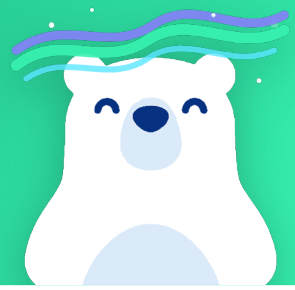


Class	Responsibility
MainActivity · Page	`MainActivity` is just the screen (Back, bar colors). The page lives in `Page`: WebView (assets at `http://localhost`), the `NanuqAndroid` bridge, 20 s heartbeat, recreate the WebView if it dies, external links http(s) only. It can start without a screen (`resumeHeadless`, with `?resume=1`) and the screen adopts it later (`MutableContextWrapper`).
NativeAudio	The music: one `MediaPlayer` per player, its own audio focus (transient loss: pause and resume; another app: pause and resume once nothing is playing, via `isMusicActive`, up to 5 min), click-free ramps when starting, pausing and releasing. Notices to the page carry a song number (`gen`) so old ones are ignored.
MediaCenter · PlaybackService	MediaSession and notification; foreground service with CPU and Wi-Fi locks while playing; while paused it stays (without locks) so the controls don't disappear, until you swipe the notification or close the app. ▶ with no page starts the page headless.
NanuqBrowserService · Resume · MediaButtonReceiver	"Resume" with the app closed: the service (answers system apps only) gives the last song to Android's interface; the receiver (`MEDIA_CONTENT_CONTROL` permission, system only) gets the headphones' ▶.
Downloads	One-at-a-time downloads, Wi-Fi only by default, limit with replacement of automatic ones; encrypted index; Opus only if its Ogg pages are complete (`OggCheck`).
Prefetch	What's next in the queue, in the cache (≤ 400 MB per file, 1 GB total, 300 MB free).
LocalServer	HTTP on 127.0.0.1 with a token: `a/` audio, `c/` cover, `p/` prefetch; Range; 10 s per request, ≤ 100 headers, 32 threads.
SecureStore	AES-GCM store with a key in Android Keystore; locked / lost states; the big indexes in their own file.
NativeHttp · Discovery	HTTP without CORS only for SomeDL routes; UDP discovery for Jellyfin and parties.

The `android-bridge.js` bridge is injected into `index.html` at build time: it replaces `navigator.mediaSession` with one that talks to `MediaCenter` and defines `NanuqNativeAudio` (used like an `<audio>`; with `blob:` or `data:`, demo mode, it uses an inner `<audio>`). Only Jellyfin-shaped URLs are downloaded (`Downloads.okUrl`) and redirects only within the same server (`openSameHost`).

11 The desktop app

Electron for Linux and Windows.



IPC channel	What it does
nanuq:discover	Finds Jellyfin servers over UDP.
nanuq:http	HTTP without CORS, SomeDL routes only, no redirects, ≤ 4 MB.
nanuq:sec (sync) · nanuq:secreset	Encrypted store: random AES-256 key saved with `safeStorage` (keyring / DPAPI); the big indexes in their own file (`SEC_BIG`).
nanuq:discord	Rich Presence over the `discord-ipc-N` socket (only sockets you own), with a minimal retry and status for Settings.
nanuq:party · nanuq:party-ev	Open/close the room and run it (jump, order, played, now, tick, lyrics).

- Every IPC channel checks `fromApp(e)` : only the page from our origin can call it.
- Window with `contextIsolation` , `sandbox` and no Node; permissions only for notifications, fullscreen and clipboard.
- Electron fuses: no `runAsNode` or `NODE_OPTIONS` , asar integrity and load only from the asar.
- **Code integrity:** no HTTP disk cache (`disable-http-cache` ; the old `Cache` folder is deleted at startup) and, before the page opens, the service worker, Cache Storage and V8 code cache are cleared (also when reloading with F5). `window.json` only accepts sensible numbers.
- Local server: GET/HEAD only, `Host` 127.0.0.1/localhost (against DNS rebinding), paths ≤ 512 and no null bytes.

12 Security

Nothing from outside is trusted.



- **CSP** without inline scripts, without `eval`, iframes or `<base>`; only `style` attributes.
- **Outside JSON** through `jparse` (no `toString` / `valueOf` / `__proto__`) and texts through `str()`: never `String(x)` on foreign data.
- **HTML**: every value goes through `esc()`; ids through `sid()` (letters, digits and dashes only) before going into a path.
- **Lyrics** stripped of control characters and tags (`cleanLyricsText`), ≤ 200,000 characters.
- **Native bridges** with route allowlists and size limits; no following redirects to another server.
- **Personal data** encrypted on disk; Android backup disabled (`allowBackup=false` and extraction rules).
- **Android, what other apps can reach**: only the “resume” service (answers system apps only) and the button receiver (requires `MEDIA_CONTENT_CONTROL`, held only by the system). The native player only opens http(s) addresses.
- **Integrity on disk**: nothing stored is used without checking. What's encrypted (personal data, downloads index, party songs) uses AES-GCM, which detects any change; unencrypted preferences are validated when read; on Android everything lives in the app's private storage and an Opus download is only kept if all its Ogg pages arrive complete with their CRC (`OggCheck.valid`; conversions send no size); the server's audio type is only accepted as `audio/…` (`OggCheck.mime`).



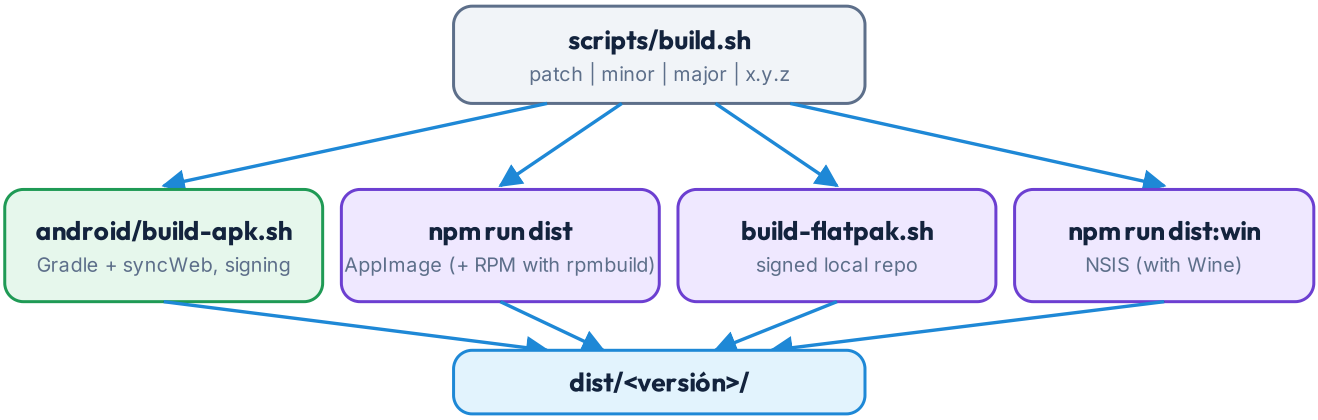
Nanuq's tip

The full detail (what is protected and known limits) is in `SECURITY.md`.

13 Building and releasing

One command for every platform.





The version lives in `web/app.js` (VERSION) and `web/sw.js` (CACHE); the APK, package.json and Flatpak take it from there.

- **Versioning**: new features bump the minor number; fixes, the last one. The APK `versionCode` is M·10000 + m·100 + p.
- **Android** needs JDK 21, the SDK, Node.js and `android/keystore.properties`. Always signed with the same key.
- **Linux**: the RPM uses the system `fpm` (`USE_SYSTEM_FPM=true`) and is only built if `rpmbuild` exists. The Flatpak is built from `linux-unpacked`.
- **Automated tests** (no dependencies; if they fail, no packages): `android/test` (OggCheck and the bridge player, in `build-apk.sh`) and `linux/test` (party encryption, who joins the room and the encrypted store, in `build.sh`).
- **Checks**: `node --check web/app.js` and, for UI, Chromium in demo mode.

14 Conventions and pitfalls

What to know before changing anything.



Rule	Why
UI changes only in `web/`	Android and Linux copy it at build time.
Every visible text in `tx('…')` + its `EN` translation	Otherwise it shows in Spanish in English mode. One full sentence per `tx`, no gluing fragments.
No `onclick=`, `<script>` or `<style>` from JS	The CSP blocks them silently.
Unique ids across the whole app	A duplicated id hid the Search box (1.30.2).
Unique `view-transition-name`	If repeated, the transition aborts with no visible error.
Endless animations only in bursts (`nqBurst`)	At 120 Hz each one costs ≈ 1 phone core.
Respect `prefers-reduced-motion`	Accessibility.
After writing to Jellyfin, show the local copy and confirm later	An immediate read may return stale data (lyrics, playlists, favorites).
`.ig-crowd` without `z-index`	It would create its own layer and everyone would end up behind the DJ.
The ProGuard rule points to `Page$Bridge`	If the bridge moves to another class and the rule doesn't, the minified APK strips the methods and the page has no bridge.
Testing on the phone: `am stop-app`, not `force-stop`	`force-stop` leaves the app “stopped” and Android won't deliver media buttons to it; `am kill` can't stop the foreground service.

15 License

GPLv3 or later, file by file.



Nanuq is free software under the **GNU General Public License, version 3 or (at the recipient's option) any later version**: `GPL-3.0-or-later`. The full text is in `LICENSE`, at the root of the repository.

What it means for whoever modifies it

- You can use, study, copy and modify the code for any purpose.
- If you **distribute** Nanuq or a modified version (an APK, a package, the PWA on a server others download from), it must go under the same license and with its source code (or an offer to get it), keeping the copyright and license notices.
- A modified version must say it is one. For Android, use another `applicationId` and your own signing key: only the author signs the official APK (see “Building and releasing”).
- It comes with **no warranty**.

How the code is marked

Every source file starts with two lines in the **SPDX** format (the one used by license tools and the REUSE specification):

```
SPDX-FileCopyrightText: 2026 Fleyrer y colaboradores de Nanuq
SPDX-License-Identifier: GPL-3.0-or-later
```

They go as a comment in each file's language (`//` , `/* */` , `<!-- -->` or `#`), after the `#!` in scripts and after the `<!DOCTYPE>` in HTML. When you create a new file, add the same two lines.

Where else it shows up

Place	How
`LICENSE`	Official GPLv3 text (GitHub detects it and shows it on the repository).
README.md · README.en.md	Line at the top, “License” section and a link in the menu.
App settings	“Nanuq x.y.z” row: the license and a link to `LICENSE` (opens in the browser).
linux/package.json	`"license": "GPL-3.0-or-later"`
Flatpak (metainfo)	`<project_license>GPL-3.0-or-later</project_license>`
These manuals	Cover and this chapter.

What isn't Nanuq's

Component	License	Where
Electron	MIT	Inside the desktop apps (Linux and Windows).
Inter · Outfit	SIL Open Font License	Fonts, loaded from Google Fonts (not in the repository).
Gradle wrapper	Apache 2.0	`android/gradlew`
Jellyfin	GPL-2.0	The server: a separate project Nanuq talks to through its API. Nanuq is not affiliated with it.

PART 2

Deep level: every function

All 892 functions and methods in the code, file by file, with the line where they start and what they do.



16 web/app.js

603 functions, in file order.



web/app.js 6,108 lines

Base: helpers and storage from line 1 · 25

Function	Line	What it does
$	5	Shortcut for document.querySelector (optionally inside another element).
$$	6	querySelectorAll shortcut that returns an array.
jparse	10	Parses outside JSON without toString/valueOf/__proto__.
str	11	Converts any value to text without ever throwing.
esc	12	Escapes text to put it into HTML (& < > " ').
sid	16	Cleans a server id: only letters, numbers and dashes.
num	17	Converts to a number; 0 if it isn't finite.
numOrNull	18	Converts to a number or returns null if empty or invalid.
ic	30	HTML for an SVG icon from the <defs> (#i-name).
fmt	31	Formats seconds as m:ss or h:mm:ss.
isSensitive	38	Does this stored key hold personal data (goes to the encrypted store)?
SEC.ok	46	Is the native encrypted store open and usable?
SEC.shut	47	Is the store locked or lost (nothing is written in plain text)?
store.get	55	Reads a stored value (encrypted if sensitive, otherwise localStorage) and migrates old plain copies.
store.set	67	Saves a value; sensitive data goes to the encrypted store or, if closed, only to memory.
store.del	78	Deletes a stored value from the store and localStorage.
secMigrate	81	Moves into the encrypted store what older versions left in plain text.
Vault.kind	95	Name of the encrypted store in use (or null).
Vault.init	98	Cold start: retries opening the store for a few seconds before giving up.
Vault.save	101	Saves the Jellyfin session.
Vault.load	102	Reads the saved session (or flags that the store is locked).

Function	Line	What it does
`Vault.reset`	109	“Start over”: sets unreadable data aside and creates a new store.
`uid`	117	Random id (UUID without dashes).
`deviceId`	118	Persistent id of this device for Jellyfin.
`deviceName`	119	System name (Android, Linux, Windows…) announced to Jellyfin.

Server addresses from line 122 · 1

Function	Line	What it does
`parseServer`	124	Parses what you typed as a server (IPv4, IPv6, name, port and path) and builds the base URL.

Jellyfin API from line 177 · 58

Function	Line	What it does
`Jellyfin.constructor`	180	Creates the Jellyfin client from the saved session.
`Jellyfin.authHeader`	181	Builds the MediaBrowser authorization header with device, version and token.
`Jellyfin.req`	185	API request with a timeout and no cache; detects 401 and reads JSON with jparse.
`Jellyfin.alt`	203	Tries several API routes until one doesn't return 404.
`Jellyfin.probe`	204	Checks that an address is a Jellyfin (/System/Info/Public).
`Jellyfin.login`	212	Logs in with username and password and stores the token.
`Jellyfin.cfg`	216	Session data to save.
`Jellyfin.map`	217	Converts a Jellyfin item to Nanuq's format, cleaning ids, numbers and texts.
`Jellyfin.items`	236	Generic /Items query with the fields the app uses.
`Jellyfin.albums`	240	Paginated, sorted Jellyfin albums.
`Jellyfin.item`	244	One item by id (with a fallback route for old servers).
`Jellyfin.album`	245	A Jellyfin album and its tracks.
`Jellyfin.artists`	249	Album artists, paginated.
`Jellyfin.artistFilters`	254	Possible filters to find an artist's items (by several ids or by name).
`Jellyfin.firstNonEmpty`	260	Tries filters in order and returns the first non-empty result.
`Jellyfin.artist`	264	An artist and their albums (built from their songs if needed).
`Jellyfin.artistTracks`	279	All of an artist's songs.
`Jellyfin.genres`	287	Music genres trying several routes; otherwise derived from albums.
`Jellyfin.genreParam`	306	Query parameters for one or more genres (by id or by name).

Function	Line	What it does
`Jellyfin.genre`	311	A genre's albums.
`Jellyfin.genreTracks`	318	A genre's songs (from the songs' tags or their albums).
`Jellyfin.playlists`	327	Your audio playlists.
`Jellyfin.playlist`	328	A playlist and its songs.
`Jellyfin.views`	332	The user's music libraries (root of Folders).
`Jellyfin.folder`	336	A folder's contents.
`Jellyfin.search`	337	Search for artists, albums and songs.
`Jellyfin.img`	344	Cover URL (local if it's downloaded on the phone).
`Jellyfin.streamUrl`	351	Universal streaming URL: formats that play as-is and conversion to Opus or MP3.
`Jellyfin.stream`	357	URL to play: download, ahead-of-time cache or streaming.
`Jellyfin.dlUrl`	361	URL to download a song to the phone (the streaming one at download quality: Opus or MP3, or the file as-is).
`Jellyfin.peaks`	362	Downloads a light copy and computes the real waveform.
`Jellyfin.tracks`	367	Paginated, sorted library songs.
`Jellyfin.lyrics`	371	A song's lyrics (Jellyfin 10.9+), normalized.
`Jellyfin.mix`	376	Songs for a mix (order, filters, genres, years).
`Jellyfin.uploadLyrics`	383	Uploads lyrics (.lrc or .txt) to Jellyfin.
`Jellyfin.searchLyrics`	390	Searches for lyrics in Jellyfin's providers (LrcLib).
`Jellyfin.pickLyrics`	394	Picks one of the found lyrics to save it.
`Jellyfin.setFavorite`	396	Marks or unmarks a favorite.
`Jellyfin.inFavorites`	402	Is it in Jellyfin's favorites section?
`Jellyfin.isFavorite`	403	Is this song a favorite according to its user data?
`Jellyfin.favPlaylist`	410	Finds the “Favorited Songs Playlist” plugin's playlist.
`Jellyfin.favorites`	417	Your favorites (IsFavorite filter), paginated and sorted.
`Jellyfin.setFavoriteLegacy`	435	Old route to save favorites (second attempt).
`Jellyfin.instantMix`	437	Jellyfin instant mix from a song (radio).
`Jellyfin.addToPlaylist`	442	Adds songs to a playlist.
`Jellyfin.refreshLibrary`	444	Asks Jellyfin to rescan the libraries (administrators).
`Jellyfin.savePlaylist`	445	Creates a new playlist with songs.
`Jellyfin.report`	446	Tells Jellyfin a song started, progressed or stopped (and reasserts favorites).
`Jellyfin.logout`	452	Logs out on the server (revokes the token).
`Jellyfin.wsUrl`	455	Jellyfin WebSocket URL.

Function	Line	What it does
`Jellyfin.postCapabilities`	456	Announces that this Nanuq accepts remote-control commands.
`Jellyfin.sessions`	458	Active sessions (for My devices).
`Jellyfin.sessionCmd`	459	Sends a playback command to another session.
`Jellyfin.sessionPlay`	460	Sends a queue to play on another session.
`Jellyfin.sessionGeneral`	461	Sends a general command (repeat, shuffle) to another session.
`Jellyfin.itemsByIds`	462	Requests several items by id, 100 at a time.
`netError`	468	Clear message when the server can't be reached.
`normLyrics`	477	Normalizes Jellyfin lyrics (synced or plain).

Audio: waveforms and synthesizer from line 489 · 6

Function	Line	What it does
`actx`	492	Shared AudioContext (created on first use).
`decode`	493	Decodes audio into an AudioBuffer.
`peaksFromBuffer`	494	Computes waveform peaks from an AudioBuffer.
`rng`	501	Seeded pseudo-random generator.
`wav`	502	Converts an AudioBuffer into a WAV file.
`synth`	509	Synthesizes a test song for demo mode.

Demo library from line 550 · 43

Function	Line	What it does
`Demo.constructor`	553	Creates the demo library (artists, albums, songs and playlists).
`Demo.cfg`	588	Demo-mode session.
`Demo.wait`	589	Simulates network latency.
`Demo.albums`	590	Demo version of Jellyfin.albums.
`Demo.album`	595	Demo version of Jellyfin.album.
`Demo.artists`	596	Demo version of Jellyfin.artists.
`Demo.artistBy`	598	Finds a demo artist by id or name.
`Demo.artist`	599	Demo version of Jellyfin.artist.
`Demo.artistTracks`	600	Demo version of Jellyfin.artistTracks.
`Demo.genres`	601	Demo version of Jellyfin.genres.
`Demo.genreTracks`	602	Demo version of Jellyfin.genreTracks.
`Demo.genre`	603	Demo version of Jellyfin.genre.
`Demo.playlists`	604	Demo version of Jellyfin.playlists.

Function	Line	What it does
`Demo.playlist`	605	Demo version of Jellyfin.playlist.
`Demo.views`	606	Demo version of Jellyfin.views.
`Demo.folder`	607	Demo version of Jellyfin.folder.
`Demo.search`	612	Demo version of Jellyfin.search.
`Demo.img`	616	Draws a generated SVG cover for demo mode.
`Demo.buf`	630	Synthesized audio for a demo song (cached).
`Demo.stream`	631	Blob URL of a demo song's WAV.
`Demo.peaks`	632	Waveform of a demo song.
`Demo.tracks`	633	Demo version of Jellyfin.tracks.
`Demo.mix`	639	Demo version of Jellyfin.mix.
`Demo.lyrics`	649	Made-up synced lyrics for a demo song.
`Demo.setFavorite`	661	Demo version of Jellyfin.setFavorite.
`Demo.isFavorite`	662	Demo version of Jellyfin.isFavorite.
`Demo.inFavorites`	663	Demo version of Jellyfin.inFavorites.
`Demo.favorites`	664	Demo version of Jellyfin.favorites.
`Demo.instantMix`	665	Demo version of Jellyfin.instantMix.
`Demo.addToPlaylist`	666	Demo version of Jellyfin.addToPlaylist.
`Demo.savePlaylist`	667	Demo version of Jellyfin.savePlaylist.
`Demo.report`	668	Does nothing (demo doesn't report plays).
`Demo.logout`	669	Does nothing (no server).
`logoInner`	674	HTML of the logo bear with the theme's accessory.
`bearHTML`	690	HTML of the animatable Nanuq (logo + expressive face).
`bearState`	692	Current bear state (dancing, happy, sad) for new bears.
`nqBurst`	698	Turns on an animation burst for a few seconds (battery saving).
`nqBurstEnd`	706	Ends the burst: animations pause and bears stay happy.
`bearDance`	711	Makes every Nanuq dance for a while.
`bearInit`	716	Puts Nanuq in the sidebar logos.
`bearMood`	721	Changes the bears' mood when music plays or pauses.
`showSplash`	732	Welcome screen with the theme's Nanuq.
`refreshBear`	743	Changes the bears' accessory when the theme changes.

Settings and themes from line 748 · 3

Function	Line	What it does
OPUS_OK	752	Does the player open Opus? Decides whether it is offered and used by default.
newAudio	755	Creates the player: on Android, the system one (NanuqNativeAudio); otherwise an <audio>.
Settings.set	785	Changes a setting and saves it.

Downloads (Android) from line 808 · 30

Function	Line	What it does
DL.init	814	Loads the phone's downloads from the native app.
DL.indexCovers	818	Index of downloaded covers by image id.
DL.coverUrl	826	Local URL of a downloaded cover.
DL.backfillCovers	828	Fills in covers and details of old or recovered downloads.
DL.config	819	Sends Android the maximum songs and whether Wi-Fi only.
DL.usable	829	Are downloads available on this device (Android with Jellyfin)?
DL.get	826	A song's download, if any.
DL.localUrl	838	Local URL of a downloaded, ready song.
DL.isLocal	839	Is this URL from the phone's local server?
DL.add	833	Requests downloading a song (manual or automatic).
DL.remove	846	Removes a song from downloads.
DL.removeAll	847	Deletes all downloads (or only automatic ones).
DL.list	848	List of downloads.
DL.bytes	849	Total space used by downloads.
DL.net	850	Current network type according to Android (wifi, cellular, none).
__nanuqDl	852	Receives a download's changes from Android and updates the view.
Plays.count	867	Times a song played on this phone.
Plays.add	868	Counts a play and auto-saves the song at 3.
PF.usable	874	Can it save ahead of time (Android with Jellyfin)?
PF.isUrl	886	Is this URL from the ahead-of-time cache?
PF.url	887	Local URL of a song already saved ahead of time (or null).
PF.soon	888	Schedules an update of the ahead-of-time window.
PF.sync	888	Computes which queue songs to save (N minutes) and asks Android.
fmtMB	904	Formats bytes as MB or GB according to the language.

Function	Line	What it does
dlErr	907	Translates an Android download error (the first number goes as {0}).
dlStatus	911	Text of a download's state.
dlStatusShort	922	Short download state for the player button.
dlSummary	928	Downloads summary (count, space, pending).
themeSummary	952	HTML of the chosen theme in Settings.
applyTheme	956	Applies a theme (colors, system bar, bears).

State and routing from line 967 · 8

Function	Line	What it does
regList	973	Registers a song list so buttons can find it.
cur	982	Current screen in the navigation stack.
go	983	Navigates to a screen (root or on top of the current one).
back	992	Goes back: closes dialogs, the player or the current screen.
cacheDrop	1002	Forgets cached views starting with a prefix.
refreshIf	1004	Redraws the current screen if it meets a condition.
playlistsChanged	1007	Forgets and re-requests a playlist that changed.
cached	1014	View cache with a cap (oldest go first).

Albums from tags from line 1017 · 15

Function	Line	What it does
albumNorm	1023	Normalizes a name to compare albums.
albumKeyOf	1025	Album key of a song from its tags (artist + album + disc).
albumGid	1032	Stable id of a tag-built album.
LIB.who	1035	Identifies the index's server and user.
LIB.index	1037	Tag-based album index (from the encrypted cache or the server).
LIB.peek	1050	The index if it's ready, without waiting.
LIB.reset	1051	Marks the index as stale so it's rebuilt.
LIB.refresh	1045	Rebuilds the index with every song.
LIB.fetchAll	1053	Downloads every song in the library (3 pages at a time).
LIB.pack	1068	Compact version of a song to save the index.
LIB.unpack	1069	Reads a compact song from the saved index, cleaning it.
LIB.save	1074	Saves the encrypted index (if it fits).
LIB.build	1079	Groups songs into albums, without duplicates.
LIB.albumsOf	1104	Albums (by tags) of a group of songs.

Function	Line	What it does
LIB.sorted	1109	Sorted album list.

Lyrics search from line 1122 · 10

Function	Line	What it does
lyrNorm	1128	Normalizes lyrics text for searching (no accents or punctuation).
LYRX.load	1131	Loads the saved lyrics index.
LYRX.add	1136	Adds a song's lyrics to the index.
LYRX.save	1141	Saves the encrypted lyrics index (debounced).
LYRX.reset	1149	Deletes the lyrics index.
LYRX.build	1151	Builds or completes the index reading lyrics 3 at a time.
LYRX.search	1177	Finds songs whose lyrics contain a text.
lyrSnippetHTML	1194	Lyrics line with the search highlighted (escaped).
lyricsHitsHTML	1199	HTML list of songs found by lyrics.
lyricsSection	1204	“In the lyrics” section that fills in while indexing.

Rendering helpers from line 1220 · 8

Function	Line	What it does
coverHTML	1222	HTML of a cover with its fallback icon.
head	1227	Screen header (title, buttons, bear, back).
albumCard	1231	An album card.
artistCard	1232	An artist card.
trackRows	1233	Container of song rows.
rowsHTML	1233	Song rows (number, title, duration, menu).
loading	1246	Loading indicator.
errBox	1247	Error box with “Retry”.

Views from line 1250 · 6

Function	Line	What it does
V.albums	1253	Albums screen (by tags, 120 at a time).
V.album	1259	An album's screen.
normTxt	1299	Normalizes text (no accents, lowercase).
mixDef	1300	A mix's definition (query by genres or years).
mixTracks	1314	A mix's songs (cached).
greeting	1315	Greeting by time of day.

DJ Nanuq from line 1317 · 85

Function	Line	What it does
djBearSVG	1323	SVG of DJ Nanuq with headphones, scarf and turntable.
djMoment	1375	Time of day and mood for the “For right now” round.
pick	1376	Picks one at random.
wpick	1378	Picks at random with weights.
wshuffle	1380	Shuffles giving more chance to heavier items.
djQid	1383	Queue id carrying the DJ session and round.
djTag	1384	Reads session and round from a DJ queue id.
djSame	1385	Are two ids from the same DJ session?
DJ.voiceOk	1388	Is there a system voice in the app's language?
DJ.save	1389	Saves the DJ session and shares it with your devices.
DJ.restore	1394	Restores the DJ session when the app opens.
DJ.sample	1401	Random sample of the library to choose genres and decades.
DJ.heavy	1407	Your 60 most played songs.
DJ.pickGenre	1413	Picks a round's genre (avoiding recent ones).
DJ.pickDecade	1423	Picks a round's decade.
DJ.rank	1435	How worn out a song is (lower = goes first).
DJ.pool	1444	Candidate songs for a round type.
DJ.build	1477	Builds a round of ~5 songs with its introduction.
DJ.next	1504	Picks and builds the next round (weights and cooldown).
DJ.items	1518	Turns a round into queue songs and records it.
DJ.lastKey	1520	Number of the last round.
DJ.start	1527	Starts a DJ session with its loading screen (here or on the device you control).
DJ.startNow	1530	Builds the first round and waits until it plays, at most 12 s (behind the loading screen).
DJ.share	1545	Leaves the DJ state in Jellyfin for another device.
DJ.handedOff	1546	The session moved to another device: it turns off here.
DJ.adopt	1565	Continues here a session coming from another device.
DJ.queueFromShare	1570	Rebuilds the DJ queue from the shared state.
DJ.queueFromTags	1572	Rebuilds the DJ queue from the queue ids.
DJ.remoteTag	1574	Is what's playing on the device you control the DJ?
DJ.shown	1575	Should the DJ be shown?
DJ.remoteTrack	1577	Fetches the rounds of the DJ playing on another device.

Function	Line	What it does
DJ.stop	1587	Turns the DJ off (the queue stays as is).
DJ.onTrack	1589	On song change: introduces the new round and adds the next.
DJ.extend	1594	Adds the next round to the end of the queue.
DJ.switchUp	1607	“Change the vibe”: another round instead of what was left.
DJ.announce	1615	Introduces a round (bubble, notice and voice).
DJ.show	1615	Shows the introduction where appropriate.
DJ.speak	1615	Reads the introduction aloud and lowers the music.
DJ.visible	1640	Is the page visible?
DJ.bears	1641	Turns the DJs' dancing on or off.
DJ.say	1645	DJ speech bubble in the player.
DJ.scene	1650	Record scene in the player (sleeve, vinyl, entries and exits).
DJ.paint	1669	Updates everything DJ-related on screen.
DJSHARE.ok	1699	Can the DJ be shared (Jellyfin and sync on)?
DJSHARE.state	1700	DJ state to share (bounded).
DJSHARE.push	1705	Saves the DJ state in /DisplayPreferences.
DJSHARE.pushRaw	1713	Saves an already built state.
DJSHARE.pushSoon	1717	Saves the state in 1.5 s.
DJSHARE.offSoon	1719	Announces the DJ turned off, if the session is still this device's.
DJSHARE.pull	1721	Reads the shared DJ state.
djClean	1726	Cleans the read DJ state (untrusted data).
djLiveSync	1744	Animates the DJ only while it's visible.
djTileHTML	1756	DJ Nanuq card on Home.
DJL.show	1766	Shows the DJ loading screen with rotating lines (a tap dismisses it).
DJL.hide	1770	Hides the loading screen (at least 1 s on screen, or right away if tapped; only once).
V.dj	1783	DJ Nanuq screen.
nanuqSceneSVG	1807	Nanuq scene for an empty Home cell.
nanuqBear	1823	Nanuq ready to place in a scene.
nanuqWideSVG	1827	Wide Nanuq scene (two cells).
nanuqWideTileHTML	1844	Wide cell with a scene.
nanuqTileHTML	1849	Cell with a scene.
nanuqVisit	1856	A Nanuq from another theme visits a cell.

Function	Line	What it does
`nanuqVisits`	1876	Schedules visits every now and then.
`fillGaps`	1884	Fills Home's grid gaps with scenes.
`tileHTML`	1897	A mix tile.
`offlineBanner`	1902	“No connection” notice with access to Downloads.
`V.home`	1907	Home screen.
`V.mix`	1952	A mix's screen.
`V.songs`	1962	Songs screen (paginated).
`V.artists`	1990	Artists screen.
`fmtLong`	1997	Long duration (“3 h 12 min”).
`V.artist`	1990	An artist's screen (songs, albums, appears on).
`V.genres`	2079	Genres screen.
`V.genre`	2084	A genre's screen.
`V.playlists`	2095	Playlists screen.
`V.playlist`	2100	A playlist's screen.
`V.favorites`	2109	Favorites screen (confirmed with Jellyfin).
`V.folders`	2135	Folders screen.
`V.search`	2144	Search screen (includes lyrics search).
`songsBlock`	2166	Songs block as a list or grid.
`V.queue`	2181	Queue tab (phone).
`V.more`	2182	More screen.
`V.settings`	2185	Settings screen.
`V.downloads`	2255	Downloads screen.
`renderDownloadsSoon`	2271	Redraws Downloads shortly.

Your year and Your week (video) from line 2277 · 36

Function	Line	What it does
`ServerStats.available`	2289	Is the Playback Reporting plugin present and readable?
`ServerStats.plays`	2297	Plays between two dates from the plugin (day by day).
`Recap.total`	2359	Total video length.
`Recap.setMode`	2355	Switches between year and week recap.
`Recap.plays`	2367	Plays for the period (plugin, basic data or demo).
`Recap.tally`	2377	Groups plays: songs, artists, genres, hours, days, streak.
`Recap.collect`	2401	Year recap data.
`Recap.collectWeek`	2413	Data for last week's recap (Monday to Sunday).

Function	Line	What it does
`Recap.jellyfinYear`	2429	Songs played in a period according to Jellyfin's basic data.
`Recap.assets`	2442	Prepares the video's images: bears, DJ, covers and palette.
`Recap.loadImg`	2464	Loads an image with limits (8 MB, reduced) for the canvas.
`Recap.sceneAt`	2480	Which scene plays at a given second of the video.
`Recap.frame`	2479	Draws a video frame (with transitions between scenes).
`Recap.transition`	2488	Transition between two scenes (slide, circle, zoom, wipe).
`Recap.bg`	2498	A scene's background: gradient, beat glow and snowflakes.
`Recap.weather`	2515	Draws the music forecast (sun, storm, snow, rainbow...).
`Recap.bear`	2532	Draws a Nanuq hopping to the beat.
`Recap.text`	2539	Text fitted to width, with a shadow.
`Recap.cover`	2549	Cover with rounded corners.
`Recap.confetti`	2555	Confetti shower.
`Recap.scene`	2563	Draws a recap scene.
`Recap.music`	2810	Video music generated with Web Audio (with a drumroll).
`Recap.stop`	2841	Stops playing or recording the video.
`Recap.play`	2840	Plays the video on the canvas and, if asked, records it (MediaRecorder).
`Recap.save`	2868	Saves the video (Android in chunks, desktop as a download).
`mp4Finalize`	2894	Writes the real duration into the fragmented MP4.
`recapCloseAudio`	2935	Closes the video's audio only once.
`ease.out`	2936	Animation curve (out).
`ease.inOut`	2936	Animation curve (in-out).
`ease.back`	2936	Animation curve with overshoot.
`clamp01`	2937	Clamps a value between 0 and 1.
`mixHex`	2938	Mixes two hex colors.
`hexA`	2939	Hex color with transparency.
`recapYearOpen`	2941	Is it the first week of December?
`recapTileHTML`	2942	“Your year with Nanuq” card on Home.
`V.recap`	2945	Your week / Your year screen (view and export).

Song requests (SomeDL) from line 2988 · 26

Function	Line	What it does
`__nanuqHttp`	2994	Receives a native SomeDL request's response (Android).
`reqNative`	2995	Is there a native bridge to talk to SomeDL without CORS?
`reqRawHttp`	2996	Request to SomeDL through the native bridge or fetch.

Function	Line	What it does
`parseSomedl`	3006	Parses the SomeDL address (port 5000 by default).
`reqNetError`	3015	Clear message when SomeDL doesn't respond.
`REQ.save`	3030	Saves the request list.
`REQ.setCfg`	3031	Saves or clears the SomeDL address.
`REQ.pending`	3032	Requests still on the way.
`REQ.visible`	3033	Is the app visible (to check more often)?
`REQ.http`	3018	Request to the SomeDL API with error handling.
`REQ.probe`	3042	Checks that an address is SomeDL.
`REQ.add`	3048	Sends a song request to SomeDL.
`REQ.schedule`	3060	Schedules the next request check.
`REQ.poll`	3061	Checks the requests' state in SomeDL.
`REQ.announce`	3090	Announces (and notifies) which requests finished.
`REQ.retry`	3105	Requests a failed song again.
`sysNotify`	3098	System notification (Android or desktop).
`askNotifyPermission`	3118	Asks for notification permission in the browser.
`reqRowHTML`	3121	A SomeDL request row.
`reqListHTML`	3139	Request list.
`renderReqSoon`	3137	Redraws the request list shortly.
`V.request`	3141	Request a song screen.
`refreshFromJellyfin`	3201	“Refresh from Jellyfin”: forgets caches, rebuilds indexes and refreshes the queue.
`sessionGone`	3224	Confirms whether a 401 really is an expired session.
`sessionExpired`	3235	Expired session: stops everything and asks to log in.
`render`	3243	Draws the current screen with a transition and error handling.

`Player` from line 3279 · 64

Function	Line	What it does
`P.pause`	3289	Pauses, marking that Nanuq requested it.
`onAudio`	3294	Listens to an event on both <audio> elements but only handles the current song's.
`saveQueue`	3296	Saves the queue and updates the ahead-of-time window.
`QPOS.save`	3301	Saves the song's second to resume it.
`QPOS.restore`	3307	Where to resume the song when the app opens.
`QPOS.clear`	3311	Forgets the saved position.

Function	Line	What it does
`setQueue`	3315	Replaces the queue and starts playing.
`addToQueue`	3316	Adds songs at the end or next.
`ptime`	3326	Current second of the song (here or on the other device).
`pdur`	3327	Song duration.
`setSrc`	3333	Changes the audio source resetting its state.
`seekTo`	3337	Seeks to a second (for live conversions it requests from there).
`playIndex`	3358	Plays song N of the queue.
`armStallWatch`	3375	Warns if a song takes too long to start.
`MP3.add`	3404	Remembers a song that must be requested as MP3.
`localFailed`	3408	The local copy failed: it plays from the server (and is downloaded again).
`canFallback`	3420	Can this song be requested as MP3 (in “Original” or with Opus)?
`fallbackMp3`	3423	Requests the song as MP3 if the original or Opus one won't start.
`armFallback`	3433	Schedules the switch to MP3 if it doesn't start in 6 s.
`playFailed`	3439	Explains why it couldn't play.
`audioDead`	3460	Is the audio dead (network error)?
`replayHere`	3461	Requests the song again from the second it was at.
`rampStop`	3474	Stops a volume ramp.
`pageHidden`	3480	Is the page out of view? (screen off or app in background): no fades or DJ crossfade then.
`rampsFinish`	3482	Finishes the running volume ramps right away (when the page gets hidden).
`ramp`	3486	Timer-driven volume ramp (in decibels, or constant-power for the crossfade).
`fadeStop`	3495	Stops a fade in progress.
`fadeTo`	3496	Fades the volume to a value.
`softMs`	3505	Pause/play fade length (0 when off).
`userVol`	3504	The volume you chose (lowered if the DJ is talking).
`pauseSoft`	3502	Pauses with a fade.
`playSoft`	3510	Resumes with a fade, or fades back up if it was fading out (reloads if another app paused).
`partyNow`	3526	Do the controls handle the party right now?
`partyHostOpen`	3527	Is a party room open on this computer?
`toggle`	3527	Play/pause (party, remote control or here).

Function	Line	What it does
next	3534	Next song (respects repeat and the end of the queue).
prev	3546	Previous song (or restart if past 3 s).
seekFrac	3547	Seeks to a fraction of the song.
removeAt	3553	Removes a song from the queue.
clearQueue	3553	Clears the queue.
shuffleQueue	3554	Shuffles the queue (the current one stays first).
setMode	3562	Changes the repeat mode.
xfDrop	3574	Releases the <audio> of the song that left.
xfKill	3572	Stops the outgoing crossfade song (on pause or song change).
xfCheck	3576	Decides whether the DJ crossfade starts (3–5 s before the end).
xfStart	3585	Crosses the ending song with the next one using both <audio> elements.
updateModeBtn	3599	Updates the repeat button.
loadPeaks	3601	Loads a song's waveform.
cssVar	3615	Reads a theme CSS variable (cached).
drawWaves	3607	Draws the progress bars or the waveform.
roundRect	3631	Rounded rectangle on canvas.
updateNow	3641	Updates everything that shows the current song.
qualPaint	3674	In the maximized player, how the song arrives (e.g. “Opus · 96 kbps”, “Original”, “On the phone”).
updatePlayState	3664	Play/pause icons and system controls state.
DISC.can	3702	Is there a Discord bridge (desktop)?
DISC.update	3702	Schedules the update to Discord.
DISC.send	3703	Sends Discord what's playing (or clears it).
dcStatusPaint	3719	Discord status line in Settings.
rcKeep	3740	Silent audio so the system keeps the controls.
sessionHold	3754	Keeps the system controls while loading or when the queue ends.
rcKeepOff	3760	Stops the silent audio.
stallCheck	3806	Detects audio stuck mid-song and requests it again.
netBack	3815	Network is back: resumes what was cut and reconnects.
paintVol	3858	Draws the volume bar and its icon.

Lyrics from line 3863 · 32

Function	Line	What it does
lrcKey	3866	Key of lyrics saved on the device.
lrcTime	3867	Formats a time as mm:ss.cc (LRC).
parseLRC	3868	Reads an LRC file (timestamps and offset).
toLRC	3882	Builds a clean LRC file.
cleanLyricsText	3893	Cleans typed or pasted lyrics (control chars, HTML, size).
lyricsFromText	3902	Converts text (LRC or plain) into lyrics.
localLyrics	3903	Lyrics saved on this device.
lyrSame	3911	Are two lyrics the same (text and timing)?
lyricsSaved	3919	Shows freshly saved lyrics until Jellyfin serves them.
getLyrics	3934	A song's lyrics (local, pending, cache or server).
setLyricsMode	3942	Opens or closes the lyrics with a transition.
setQueueMode	3955	Opens or closes the player's queue with a transition.
renderSheetQueue	3957	Queue list inside the player.
applyLyricsMode	3947	Applies lyrics mode without a transition.
reloadLyrics	3983	Reloads the current lyrics.
renderLyrics	3981	Draws the current song's lyrics.
estimateTimes	3999	Spreads approximate timing for unsynced lyrics.
syncLyrics	3987	Highlights the playing line and scrolls the lyrics.
refreshLyricsButtons	4051	Lyrics or Find lyrics button depending on whether there are lyrics.
syncStart	4066	Opens the editor to sync by hand.
syncEnd	4080	Closes the editor.
syncCancel	4084	Cancels syncing.
renderSync	4048	Draws the sync editor.
syncEditorTick	4092	Marks the playing line in the editor.
syncMark	4099	Marks the start of the highlighted line.
syncUndo	4112	Undoes the last mark.
syncNudge	4120	Moves the last mark ¼ s.
syncSave	4129	Fills in timing and saves the LRC.
saveLyricsText	4143	Saves lyrics in Jellyfin (or on the device if it can't).
lyricsSearch	4158	Searches LrcLib for lyrics and lets you choose one.
lyricsPaste	4188	Dialog to paste or edit lyrics.
forgetLocalLyrics	4206	Removes the lyrics saved on this device.

Queue from line 4209 · 5

Function	Line	What it does
renderQueue	4211	Draws the queue panel.
queuePanel	4222	Draws the side queue panel only if visible (wide screen or open).
renderQueueAll	4223	Redraws every queue view.
moveQueue	4228	Moves a song in the queue (also in remote control).
markCurrent	4280	Highlights the playing song in lists.

Toasts, menus and dialogs from line 4283 · 2

Function	Line	What it does
toast	4285	Brief on-screen notice.
menu	4286	Options menu (bottom sheet).

My devices from line 4294 · 63

Function	Line	What it does
devLabel	4299	Friendly name of a device.
devName	4301	Device name for sentences.
expand6	4303	Expands an IPv6 address.
ipKey	4309	Network and privacy of an IP address.
sameNetwork	4317	Are two addresses on the same network?
RC.on	4299	Is My devices turned on?
RC.start	4330	Starts My devices (WebSocket and polling).
RC.stop	4330	Stops My devices.
RC.connect	4330	Opens Jellyfin's WebSocket with retries.
RC.send	4350	Sends a message over the WebSocket.
RC.onMessage	4342	Handles WebSocket messages (commands, favorites, keep-alive).
RC.fromMe	4355	Does the command come from your own user?
RC.onPlaystate	4354	Runs received pause, next, seek...
RC.onPlay	4355	Runs a play command (and adopts the DJ if it comes with it).
RC.onGeneral	4404	Runs received repeat or shuffle.
RC.queueForReport	4411	Queue reported to Jellyfin.
RC.reportSoon	4380	Reports the state to Jellyfin shortly.
RC.schedule	4418	Schedules the next device poll.
RC.poll	4418	Looks for your other Nanuqs and follows the one you control.

Function	Line	What it does
RC.maybeFollow	4439	Connects automatically to another Nanuq playing on your network.
RC.enter	4445	Starts controlling another device.
RC.leave	4436	Stops controlling and restores your queue.
RC.apply	4438	Mirrors here what's playing on the other device.
RC.pos	4491	Estimated second of the song on the other device.
RC.tick	4500	Updates bar, lyrics and system controls in remote control.
RC.decorate	4511	Shows “Playing on …” in the player.
RC.fail	4517	Warns that a command couldn't be sent.
RC.cmd	4518	Sends a command to the playing device.
RC.seek	4524	Seeks on the other device.
RC.window	4525	Queue window that is sent (≤ 200).
RC.playQueue	4526	Sends a queue to play there.
RC.add	4535	Adds songs to the queue there.
RC.reorder	4545	Sends the reordered queue (keeping the DJ).
RC.general	4556	Sends a general command there.
RC.here	4558	“Play here”: brings the music (and the DJ) to this device.
RC.there	4581	Sends the music (and the DJ) to another device.
__nanuqTick	4600	20-second heartbeat from Android (WebSocket, party, stuck audio).
__nanuqVis	4608	Android reports whether the app is visible.
openDevices	4609	My devices menu.
updateFavBtn	4640	Player's favorite button.
setFavLocal	4646	Marks the favorite in the local queue.
favPending	4653	Remembers a recent favorite change.
favMerge	4654	Applies recent changes over the server's list.
favHold	4672	Protects the playing song's favorite (Jellyfin 12 bug).
favHeld	4675	Is this favorite being protected?
favReassert	4676	Saves the favorite again after each playback report.
favRemote	4684	Handles favorite changes made elsewhere.
refreshFav	4706	Confirms with the server whether the song is a favorite.
toggleFav	4717	Marks or unmarks the favorite and verifies it.
favConfirm	4731	Second confirmation against the favorites section.
updateDlBtn	4751	Player's Download button.

Function	Line	What it does
dlAddToast	4760	Notice when requesting a download.
dlTap	4769	Tap on Download (request or manage).
connTest	4782	“Test connection”: measures API, cover, audio, seeking and player.
startRadio	4910	Radio from a song.
addToPlaylistMenu	4924	Menu to add to a playlist (or create one).
songOptions	4939	Common song options.
queueMenu	4946	Menu for a queue song.
trackMenu	4959	Menu for a song in a list.
askConfirm	4967	Confirmation dialog.
askName	4973	Dialog to type a name.
openSheet	4981	Opens the full-screen player.
closeSheet	4989	Closes the full-screen player.

Party (watch party) from line 4992 · 65

Function	Line	What it does
partyHost	4999	Can this device host (desktop)?
partyGuest	5000	Is this device a guest (Android)?
partyOn	5001	Is the party available on this device?
partySongUrl	5003	URL to send a song to the party (streaming at 320 kbps, converted to MP3 if needed).
partyCover	5006	240 px JPEG thumbnail of the cover.
partyCoverOk	5016	Does the cover come from the room (127.0.0.1 with token)?
partyClean	5019	Cleans the state sent by the computer (untrusted).
stopForParty	5031	Stops your music when opening the room.
PH.active	5046	Does the party handle the controls?
PH.init	5047	Prepares the party player and listens to main.js.
PH.onEv	5056	Handles room events (state, joined, left).
PH.apply	5057	Applies the room state (computer party mode).
PH.list	5072	Room song list.
PH.open	5042	Opens the room.
PH.close	5084	Closes the room.
PH.jump	5086	Plays a song from “Up next” right now.
PH.order	5092	Changes the list order.
PH.next	5075	Follows the list: plays the next ready song.

Function	Line	What it does
PH.play	5099	Plays a party song.
PH.stopCur	5073	Stops the current party song.
PH.done	5117	A song ended: deletes it and moves on.
PH.toggle	5118	Pauses or resumes the party.
PH.yieldTo	5131	You played your music: the party pauses and hands over the controls.
PH.media	5107	System controls with the party song.
PH.report	5150	Tells the room what's playing.
PH.loadLyrics	5155	Loads the party song's lyrics.
PH.line	5165	Current and next lyrics line.
PH.tick	5153	Karaoke: every second, time and line to the phones.
PG.connect	5183	Joins a room (with code or key) and reconnects on its own.
PG.leave	5184	Leaves the party.
PG.joined	5220	Are you at a party?
PG.ui	5187	Phone party mode (hides bars and player).
PG.wake	5230	The phone woke up: rejoins if needed.
PG.resume	5236	When opening the app, returns to the ongoing party.
PG.abortUp	5214	Cancels the upload in progress.
PG.onMsg	5242	Handles room messages.
PG.add	5269	Sends a song to the party (lyrics, cover and audio in chunks).
PG.remove	5325	Cancels a song that's still arriving.
PG.find	5327	Finds rooms on the network (UDP from Android).
partyAddr	5347	Converts the typed address into ws://.../fiesta.
iglooHTML	5358	Igloo HTML.
iglooBear	5374	A guest's Nanuq in their theme's costume.
iglooHoldSVG	5381	Cover held up with sparkles (who picked the song).
iglooHeldOnTop	5392	Puts the object the Nanuq was holding on top of the cover.
partyGuestColor	5403	Label color from the guest's theme.
partyCoverSrc	5411	Cover URL of a party song.
iglooSync	5419	Places guests in the circle without redrawing.
iglooLive	5451	Animates the igloo only while visible.
iglooQueue	5458	“Up next” row in full screen.
iglooQueueEdge	5467	Gradient if there are more songs out of view.
partyVolumeStep	5546	Volume with ↑ ↓ in full screen and its notice.

Function	Line	What it does
iglooIdle	5559	Hides the pointer after 2.5 s idle.
iglooVinyl	5569	Sleeve and vinyl record of the playing song.
iglooLine	5581	Lyrics on the ceiling with a sparkle.
renderPartySoon	5588	Redraws the party shortly.
partyPaintTick	5592	Updates the karaoke (line, next, time).
partyPaintUp	5598	Progress of the song you're sending.
partyRowsHTML	5599	Party song list.
partyGuestsHTML	5608	Guest list.
partyNowHTML	5609	What's playing at the party.
paintParty	5590	Draws the party screen (host or guest).
partySongsHTML	5675	Songs to send.
partySearch	5676	Party search (playlists, artists, albums, songs, lyrics).
partyBrowse	5704	Songs of an artist, album or playlist in the party.
V.party	5716	Party screen.

Global actions from line 5733 · 1

Function	Line	What it does
d0	5735	Shortcut for an element's dataset.

Login from line 5894 · 11

Function	Line	What it does
recentServers	5897	Servers used before.
rememberServer	5898	Remembers a server.
forgetServer	5904	Forgets a server.
probeTimed	5906	Tests a server measuring how long it takes.
canDiscover	5917	Can servers be discovered on the network?
discoverServers	5919	Discovers Jellyfin on the network (UDP 7359).
serverList	5938	Server list with their status.
cleartextRisk	5965	Would the password travel unencrypted over the internet?
showLogin	5954	Login screen.
startApp	6036	Starts the app with the session (queue, position, DJ, Home).
logout	6057	Logs out and deletes personal data.

17 Rest of the page

Languages and service worker.



web/i18n.js

1,040 lines · 1 functions

Function	Line	What it does
tx	1031	Translates a Spanish text into the chosen language and fills in the {0}, {1}... values.

web/sw.js

45 lines · 3 functions

Function	Line	What it does
install	8	Caches the app's files when the service worker installs.
activate	12	Deletes caches from older versions.
fetch	16	Serves only the app's files (network first for HTML/JS/CSS); the API and music go straight to the network.

18 Android

Native Java and the JavaScript bridge.



android/app/src/main/java/com/nanuq/app/Discovery.java

78 lines · 3 functions

Function	Line	What it does
`Discovery.Discovery`	27	Private constructor (static methods only).
`Discovery.find`	30	Discovers over UDP (Jellyfin or parties) and returns the answers as JSON.
`Discovery.send`	72	Sends the question to every broadcast address.

android/app/src/main/java/com/nanuq/app/Downloads.java

595 lines · 44 functions

Function	Line	What it does
`Downloads.get`	43	Single instance of downloads.
`Downloads.Downloads`	72	Sets up folder, index, local server and network watch.
`Downloads.audio`	83	Audio file for the local server.
`Downloads.audioMime`	84	Audio type for the local server.
`Downloads.cover`	85	Cover for the local server.
`Downloads.prefetch`	86	Ahead-of-time song for the local server.
`Downloads.prefetchMime`	87	Type of the ahead-of-time song.
`Downloads.onAvailable`	91	Network is back: retries downloads.
`Downloads.onCapabilitiesChanged`	92	Network changed: checks the download queue.
`Downloads.setListener`	97	Connects the page to receive changes.
`Downloads.configure`	101	Maximum songs and Wi-Fi only.
`Downloads.base`	117	Local server address.
`Downloads.okUrl`	127	Is the URL shaped like a Jellyfin audio or cover?
`Downloads.openSameHost`	141	Opens a connection following redirects only to the same server.
`Downloads.enqueue`	162	Adds a download (or updates it).
`Downloads.setScore`	196	Updates how many times a download played.
`Downloads.remove`	202	Removes a download.
`Downloads.removeAll`	208	Removes all (or only automatic ones).
`Downloads.list`	214	List of downloads as JSON.

Function	Line	What it does
`Downloads.network`	222	Current network (wifi, cellular, none), looking beneath a VPN.
`Downloads.underlying`	236	Real network beneath a VPN.
`Downloads.file`	245	File of a ready download.
`Downloads.coverFile`	253	A download's cover.
`Downloads.setCover`	262	Requests an old download's cover.
`Downloads.fetchCoverLater`	270	Downloads the cover in the background.
`Downloads.fetchCover`	273	Downloads the cover (≤ 15 MB).
`Downloads.mime`	301	A download's audio type.
`Downloads.retryAndKick`	309	Retries failed ones and continues.
`Downloads.kick`	316	Starts the download queue.
`Downloads.kickLocked`	318	Starts the queue (already locked).
`Downloads.loop`	324	Downloads pending ones one at a time.
`Downloads.nextPending`	344	The next one (yours first).
`Downloads.download`	354	Downloads a song with limits and free space; Opus only if complete (OggCheck).
`Downloads.victim`	431	The least-played automatic one (to make room).
`Downloads.removeEntry`	440	Deletes a download and its files.
`Downloads.caps`	454	Active network capabilities.
`Downloads.unmetered`	459	Is the network unmetered (Wi-Fi, Ethernet)?
`Downloads.networkAllowed`	464	Can it download on this network?
`Downloads.safe`	472	Safe file name for an id.
`Downloads.toJson`	474	Download as JSON for the page.
`Downloads.emit`	492	Notifies the page of a change.
`Downloads.ensureLoaded`	498	Reads the index when the store opens.
`Downloads.load`	500	Reads the encrypted index (or recovers from files).
`Downloads.save`	566	Saves the encrypted index.

`android/app/src/main/java/com/nanuq/app/LocalServer.java` 184 lines · 7 functions

Function	Line	What it does
`LocalServer.LocalServer`	46	Opens the server on 127.0.0.1 with a random token.
`LocalServer.base`	60	Base address with the token.
`LocalServer.acceptLoop`	62	Accepts connections (max. 32 at a time).
`LocalServer.readLine`	74	Reads a request line with a cap and a deadline.
`LocalServer.handle`	86	Handles a request (10 s and ≤ 100 headers).
`LocalServer.status`	129	Answers a status code.

Function	Line	What it does
`LocalServer.serveFile`	135	Serves a file with ranges (seeking within the song).

android/app/src/main/java/com/nanuq/app/Page.java

453 lines · 44 functions

Function	Line	What it does
`Page.run`	82	Heartbeat every 20 s to the page.
`Page.create`	143	Creates and configures the WebView (no file access, bridge) and connects music, controls and downloads.
`Page.recreate`	185	Recreates the WebView if the system killed it.
`Page.dispatchMedia`	131	Passes a notification button to the page.
`Page.showNotice`	214	“Requested song ready” notification.
`Page.setMetadata`	242	Bridge: title, artist, album and cover.
`Page.setPlaybackState`	243	Bridge: playing or paused.
`Page.setActions`	244	Bridge: available buttons.
`Page.setPosition`	245	Bridge: position and duration.
`Page.dlList`	264	Bridge: download list.
`Page.dlEnqueue`	265	Bridge: download a song.
`Page.dlCover`	266	Bridge: request a cover.
`Page.dlBase`	267	Bridge: local server address.
`Page.dlRemove`	268	Bridge: remove a download.
`Page.dlRemoveAll`	269	Bridge: remove downloads.
`Page.dlConfig`	270	Bridge: download settings.
`Page.dlScore`	271	Bridge: plays of a download.
`Page.dlNetwork`	272	Bridge: network type.
`Page.pfSet`	275	Bridge: ahead-of-time window.
`Page.pfReady`	276	Bridge: is that song already saved?
`Page.pfBroken`	277	Bridge: that saved song didn't play.
`Page.http`	280	Bridge: CORS-free request to SomeDL.
`Page.secKind`	288	Bridge: encrypted store state.
`Page.secGet`	289	Bridge: read an encrypted value.
`Page.secSet`	290	Bridge: save an encrypted value.
`Page.secDel`	291	Bridge: delete an encrypted value.
`Page.secReset`	293	Bridge: start over.
`Page.videoBegin`	297	Bridge: starts saving a video (MP4/WebM).
`Page.videoChunk`	309	Bridge: a chunk of the video (≤ 300 MB).
`Page.videoEnd`	321	Bridge: finishes and saves in Movies › Nanuq.

Function	Line	What it does
`Page.videoShare`	363	Bridge: share the video just saved.
`Page.notice`	372	Bridge: system notification.
`Page.partyDiscover`	375	Bridge: find parties over UDP.
`Page.discover`	383	Bridge: find Jellyfin over UDP.
`Page.onRenderProcessGone`	401	The WebView process died: it's recreated.
`Page.onPageFinished`	395	Without a screen: tells the page it is out of view.
`Page.shouldInterceptRequest`	407	Serves the app from the assets at http://localhost.
`Page.shouldOverrideUrlLoading`	424	Only http(s) links go out to the browser.
`Page.mime`	297	Type of an asset file.
`Page.attach`	98	The screen adopts the page (the headless one or a new one).
`Page.detach`	115	The screen closes: the page goes with it, unless music is playing.
`Page.resumeHeadless`	129	▶ with Nanuq closed: starts the page headless and continues the queue.
`Page.played`	140	It played after a headless resume: the page no longer shuts itself down at 30 s.
`Page.startOutside`	207	Opens something outside the app (browser, share), also without a screen.

android/app/src/main/java/com/nanuq/app/MainActivity.java

79 lines · 6 functions

Function	Line	What it does
`MainActivity.onCreate`	24	Shows the page (the existing headless one or a new one) and asks for notification permission.
`MainActivity.onBackPressed`	39	Back: the page decides or the app goes to the background.
`MainActivity.onStart`	48	The app came back into view.
`MainActivity.onStop`	51	The app went to the background (saves the store).
`MainActivity.onDestroy`	54	Lets go of the page: it goes with the screen unless the system closes it while music plays.
`MainActivity.setBarColor`	59	System bar color according to the theme.

android/app/src/main/java/com/nanuq/app/MediaCenter.java

399 lines · 34 functions

Function	Line	What it does
`MediaCenter.get`	50	Single instance of the media center.
`MediaCenter.MediaCenter`	70	Creates the MediaSession and the notification channel.
`MediaCenter.onPlay`	81	System play button.
`MediaCenter.onPause`	82	System pause button.

Function	Line	What it does
`MediaCenter.onStop`	83	System stop button.
`MediaCenter.onSkipToNext`	84	System next button.
`MediaCenter.onSkipToPrevious`	85	System previous button.
`MediaCenter.onSeekTo`	86	Seek from the system.
`MediaCenter.onFastForward`	87	Fast-forward from the system.
`MediaCenter.onRewind`	88	Rewind from the system.
`MediaCenter.attach`	99	Connects the page.
`MediaCenter.detach`	101	Disconnects the page.
`MediaCenter.hasPage`	104	Is there a page to receive commands?
`MediaCenter.stopIfNoPage`	107	Without a page, removes the notification.
`MediaCenter.dispatch`	116	Sends an action to the page.
`MediaCenter.setMetadata`	144	Song details (and loads the cover).
`MediaCenter.setPlaybackState`	168	Playback state.
`MediaCenter.setActions`	180	Available buttons.
`MediaCenter.setPosition`	188	Position and duration.
`MediaCenter.isPlaying`	202	Is it playing?
`MediaCenter.currentPosition`	204	Estimated current position.
`MediaCenter.publishMetadata`	209	Publishes the details on the MediaSession.
`MediaCenter.publishState`	219	Publishes the state and buttons.
`MediaCenter.refresh`	236	Adjusts service, notification and locks.
`MediaCenter.shutdown`	266	Stops everything when the app closes.
`MediaCenter.buildNotification`	277	Builds the playback notification.
`MediaCenter.action`	43	A notification button.
`MediaCenter.serviceIntent`	311	A button's intent to the service.
`MediaCenter.contentIntent`	319	Open the app from the notification.
`MediaCenter.onReceive`	327	Headphones unplugged: pause.
`MediaCenter.setNoisyReceiver`	332	Listens for headphone disconnection.
`MediaCenter.loadBitmap`	346	Loads the cover with limits (8 MB, reduced).
`MediaCenter.readCapped`	391	Reads with a byte cap.
`MediaCenter.holdForResume`	128	When resuming with Nanuq closed: the service goes foreground right away, with the last song paused.

android/app/src/main/java/com/nanuq/app/MediaButtonReceiver.java 31 lines · 1 functions

Function	Line	What it does
MediaButtonReceiver.onReceive	18	With Nanuq closed, ▶ from headphones or the system wakes it and continues the queue.

android/app/src/main/java/com/nanuq/app/Resume.java 69 lines · 2 functions

Function	Line	What it does
Resume.save	28	Saves the last song (encrypted) for the resume card.
Resume.clear	35	Deletes the last song (on logout).

android/app/src/main/java/com/nanuq/app/NanuqBrowserService.java 72 lines · 2 functions

Function	Line	What it does
NanuqBrowserService.onGetRoot	37	Answers Android's interface (system apps only) if there is something to resume.
NanuqBrowserService.onLoadChildren	43	The last song, as a playable item.

android/app/src/main/java/com/nanuq/app/NativeHttp.java 84 lines · 5 functions

Function	Line	What it does
NativeHttp.NativeHttp	19	Private constructor (static methods only).
NativeHttp.Result	24	Response (status and text).
NativeHttp.allowed	30	Allowed SomeDL route?
NativeHttp.fetch	37	CORS-free request only to SomeDL, no redirects.
NativeHttp.readAll	71	Reads the response (≤ 4 MB).

android/app/src/main/java/com/nanuq/app/PlaybackService.java 108 lines · 7 functions

Function	Line	What it does
PlaybackService.onCreate	32	Prepares the CPU and Wi-Fi locks.
PlaybackService.onStartCommand	47	Foreground and notification buttons.
PlaybackService.setLocks	68	Takes or releases the locks.
PlaybackService.release	83	Leaves the foreground and ends.
PlaybackService.onTaskRemoved	91	Removed from recents: closes everything.
PlaybackService.onDestroy	99	Releases the locks.
PlaybackService.onBind	106	No binding (started service).

android/app/src/main/java/com/nanuq/app/Prefetch.java 222 lines · 16 functions

Function	Line	What it does
Prefetch.get	40	Single instance of the ahead-of-time cache.
Prefetch.Prefetch	61	Empties the previous cache and watches the network.

Function	Line	What it does
`Prefetch.onAvailable`	71	Network is back: keeps downloading.
`Prefetch.set`	80	Receives the desired window and deletes what's left over.
`Prefetch.broken`	109	A saved one didn't play: deleted and not repeated.
`Prefetch.ready`	116	Is that song ready?
`Prefetch.file`	118	File of a ready song.
`Prefetch.mime`	125	Audio type.
`Prefetch.drop`	130	Removes a song and its file.
`Prefetch.deleteFiles`	132	Deletes a song's files.
`Prefetch.kick`	139	Starts the downloads.
`Prefetch.kickLocked`	141	Starts (already locked).
`Prefetch.loop`	147	Downloads pending ones one at a time (respects the total cap).
`Prefetch.online`	160	Is there a network?
`Prefetch.fetch`	166	Downloads a song with limits; Opus only if complete (OggCheck).
`Prefetch.trim`	211	If over the total cap, drops the farthest ones.

`android/app/src/main/java/com/nanuq/app/SecureStore.java` 234 lines · 16 functions

Function	Line	What it does
`SecureStore.get`	44	Single instance (or reads a value if given a key).
`SecureStore.SecureStore`	66	Opens the store when created.
`SecureStore.open`	77	Opens key (Keystore) and file: ready, locked or lost.
`SecureStore.kind`	52	State (retries if it was locked).
`SecureStore.reset`	125	Start over: sets the file aside and creates a new key.
`SecureStore.ready`	138	Is it open?
`SecureStore.set`	154	Saves a value (≤ 8 MB).
`SecureStore.del`	162	Deletes a value.
`SecureStore.validKey`	169	Valid key?
`SecureStore.scheduleSave`	171	Saves in 400 ms.
`SecureStore.flush`	177	Saves now.
`SecureStore.saveNow`	179	Encrypts and writes the file.
`SecureStore.encrypt`	203	Encrypts with AES-256-GCM.
`SecureStore.decrypt`	213	Decrypts and verifies.
`SecureStore.readAll`	220	Reads a file (≤ 64 MB).
`SecureStore.writeAtomic`	228	Writes atomically.

android/bridge/android-bridge.js 254 lines · 12 functions

Function	Line	What it does
abs	17	Turns a relative URL into an absolute one.
MediaMetadata	19	MediaMetadata replacement for the page.
bestArt	28	Chooses the best cover (≤ 512 px) for the notification.
mediaSession.metadata	42	Sends title, artist, album and cover to Android.
mediaSession.playbackState	47	Sends the playback state to Android.
mediaSession.setActionHandler	54	Registers the buttons the page handles.
mediaSession.setPositionState	58	Sends the position only when it drifts.
__nanuqMedia	233	Runs a notification or headphone button on the page.
sendTheme	244	Sends the theme color to the system bars.
NanuqNativeAudio	219	Object used like an <audio> that plays with the system player (or an inner <audio> in demo mode).
NanuqNativeAudio._ev	182	Turns each phone notice into <audio> events.
__nanuqAudio	220	Receives NativeAudio notices; ignores those from a previous song.

android/build-apk.sh 27 lines · 1 functions

Function	Line	What it does
(script)	1	Builds the APK and leaves it in dist/<version>/.

android/app/src/main/java/com/nanuq/app/OggCheck.java 81 lines · 2 functions

Function	Line	What it does
OggCheck.valid	47	Checks that an Ogg (Opus) file has all its pages complete with correct CRC; end of stream (EOS) only if the server didn't send the size.
OggCheck.mime	75	Accepts the server's audio type only in the audio/... form.

android/app/src/main/java/com/nanuq/app/NativeAudio.java 432 lines · 10 functions

Function	Line	What it does
NativeAudio.doSrc	131	Prepares a song in a MediaPlayer (http/https only) with its notices.
NativeAudio.doPlay	183	Requests audio focus and starts playing.
NativeAudio.onAudioFocusChange	353	What to do when another app takes or returns the audio.
NativeAudio.pauseForLoss	373	Fades out and pauses what was playing on focus loss.
NativeAudio.resumeLost	396	Resumes what focus loss paused, fading in.
NativeAudio.resumeCheck	408	After losing focus: if nothing plays on the phone anymore (≈ 4 s), resumes (up to 5 min).
NativeAudio.releaseAll	115	Stops and frees all players (the page left or reloaded).

Function	Line	What it does
`NativeAudio.stopSoft`	235	Releases the playing song fading it to 0 in 80 ms (cutting it abruptly clicked).
`NativeAudio.deClick`	248	Raises (60 ms on start) or lowers (before pausing) a song's volume without clicks.
`NativeAudio.ramp`	259	Volume ramp on the main thread (used by focus, anti-click and soft release).

19 Desktop

Electron and its packaging.



linux/main.js

865 lines · 63 functions

Function	Line	What it does
serve	47	Hardened internal 127.0.0.1:47863 server that hands the app to the window (and the party).
broadcastOf	79	Broadcast address of an IPv4 network.
discover	80	Discovers Jellyfin servers over UDP 7359.
fromApp	100	Does the IPC message come from the app's page?
vaultBackend	132	Which system keyring is available (GNOME Keyring, KWallet, DPAPI, Keychain).
secFile	148	Path of the encrypted data file.
secKeyFile	149	Path of the data key encrypted with the keyring.
writeAtomic	157	Writes a file atomically (0600).
secInit	160	Opens the encrypted store (AES-256-GCM): locked, lost or ready.
secSeal	155	Encrypts a text with AES-256-GCM (iv, tag and data).
secOpen	156	Decrypts and checks a store file (fails if altered).
secFlush	196	Writes the encrypted store to disk.
dcSet	239	Records the Discord connection state.
dcPaths	240	Possible Discord sockets or pipes (only yours).
dcWrite	251	Sends a message in Discord's format.
dcRetry	252	Looks for Discord again in 10 s.
dcConnect	253	Connects to Discord trying each socket and greets it.
dcData	269	Reads Discord's messages with limits (READY, ERROR, close, ping).
dcFlush	280	Sends the activity respecting Discord's rate (and the minimal one if rejected).
dcActivity	303	Builds the clean, bounded “Listening” activity.
jparse	340	Parses outside JSON without toString/valueOf/__proto__.
ptStr	341	Converts to safe, bounded text (no control chars).
ptIp	342	IP address without the ::ffff: prefix.

Function	Line	What it does
`ptLocal`	346	Is the address on your local network?
`ptOpenNow`	353	Is the room open?
`ptEmit`	354	Notifies the page of a room event.
`ptBase`	359	Folder on disk where the room's songs are stored (on Linux, ~/.cache/nanuq; it used to be the temp folder, which is RAM in the Flatpak).
`ptCleanTemp`	364	Deletes temporary folders from earlier rooms.
`ptNonce`	377	AES-GCM nonce for a block: the song's 8 random bytes + the block number.
`ptSeal`	378	Encrypts and authenticates a 64 KB block of a party song with the room key.
`ptEncSize`	383	Exact size the encrypted file must have (detects truncated or extended files).
`ptOpenRange`	384	Reads, checks and decrypts only the blocks of a requested range; fails if a block was altered.
`ptSniff`	400	Is it audio? (by its first bytes).
`ptCover`	413	Validates the cover a phone sends (JPEG/PNG/WebP ≤ 150 KB).
`ptPublic`	422	Public data of a song in the list.
`ptGuests`	426	Present and away guests.
`ptAddrs`	427	This computer's IPv4 addresses.
`ptStatus`	432	Full room state for the page.
`ptState`	438	Broadcasts the state to everyone (batched in 150 ms).
`ptStateNow`	439	Sends the state now.
`ptKill`	445	Removes a song from the list and the disk.
`ptFrame`	454	Writes a WebSocket frame.
`ptSend`	450	Sends a JSON message to a phone.
`ptDrop`	462	Closes a phone's connection (with a reason).
`ptGone`	468	A phone disconnected (stays away or left).
`ptUpgrade`	481	Accepts the WebSocket connection (local network only, limits).
`ptData`	496	Reads WebSocket frames with every validation.
`ptMsg`	525	Handles a message from a phone.
`ptNewCode`	547	Changes the room code.
`ptHello`	539	A phone joining: code, key, brute force.
`ptAdd`	581	A phone announces a song (with limits).
`ptChunk`	599	Receives a piece of the song and writes it to disk.

Function	Line	What it does
ptEnd	617	The song finished arriving: checks it's audio.
ptServe	634	Serves the room's songs and covers to the window (with Range).
ptGuestCover	662	Serves covers to the phones (only with their key).
ptGc	671	Cleanup every 30 s (old, half-arrived, away, silent).
ptListen	681	Opens the room port (47864 or a free one).
ptOpen	694	Opens the room: server, UDP, code and token.
ptClose	721	Closes the room and cuts every connection.
stateFile	781	File with the window's size and position.
loadState	783	Reads the window's size and position.
saveState	788	Saves the window's size and position.
start	794	Starts the app: store, server, window, permissions and protections.

linux/preload.js

25 lines · 6 functions

Function	Line	What it does
nanuqDesktop.discover	7	Bridge: discover Jellyfin on the network.
nanuqDesktop.http	8	Bridge: CORS-free request to SomeDL.
nanuqDesktop.discord	11	Bridge: Discord activity (or its status).
nanuqDesktop.party.op	14	Bridge: party operations (open, close, jump, order, karaoke...).
nanuqDesktop.party.on	15	Bridge: listen to room events.
nanuqDesktop.secure.*	17	Synchronous bridge to the encrypted store (kind, get, set, del, reset).

linux/prepare-web.js

16 lines · 1 functions

Function	Line	What it does
(script)	3	Copies web/ to www/ and takes the version from app.js.

linux/collect.js

15 lines · 1 functions

Function	Line	What it does
(script)	3	Moves the built packages to dist/<version>/.

linux/flatpak/build-flatpak.sh

54 lines · 1 functions

Function	Line	What it does
(script)	1	Builds the Flatpak, publishes it to the signed local repository and installs or updates it.

20 Build

The script that builds everything.



scripts/build.sh

73 lines · 1 functions

Function	Line	What it does
(script)	1	Bumps the version and builds Android, Linux, Flatpak and Windows.

Made with care (and lots of ice)

If you change something, remember: the interface lives in web/ and every text goes through tx().

